

Trigonometric Interpolation & Fourier Analysis

Till now we always used polynomials to approximate function, but they are not that expressive when it comes to approximating functions that are periodic.

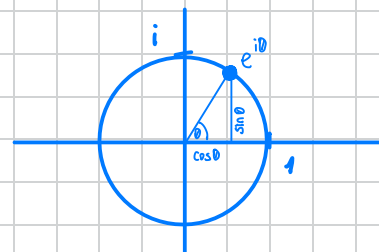
In this chapter we would like to build our functions out of oscillations of different frequencies:

$$1, e^{2\pi i t}, e^{-2\pi i t}, e^{4\pi i t}, e^{-4\pi i t}$$

Reminder: Euler's formula

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

$$\Rightarrow e^{2\pi i k t} = \cos(2\pi k t) + i \cdot \sin(2\pi k t)$$



$k \dots$ frequency; larger $|k|$ means faster oscillation (#full rotations)
 $2\pi \dots$ converts cycles into radians, because sine/cosine make one complete cycle over 2π
 $t \dots$ where you are in function's domain

Trigonometric Polynomials

A trigo. polynomial of degree at most m is

$$p_m(t) = \sum_{j=-m}^m \gamma_j e^{2\pi i j t}, \quad \gamma_j \in \mathbb{C}$$

$\rightarrow$ Each term $e^{2\pi i j t}$ represents one frequency j

Remark 3.1.2 Periodicity

Every trigo. polynomial above is automatically 1-periodic:

$$p_m(t+1) = p_m(t)$$

$$\Rightarrow \text{Take one term } e^{2\pi i j(t+1)} = e^{2\pi i j t} e^{2\pi i j} \text{ and } j \in \mathbb{Z} \\ = e^{2\pi i j t} \\ = p_m(t)$$

$\Rightarrow$ This holds for all terms individually, therefore it holds for the sum (= trigo. polynomial) as well.

For real-valued functions, we can write $p_m(t)$ as:

$$p_m(t) = \frac{a_0}{2} + \sum_{j=1}^m (a_j \cos(2\pi j t) + b_j \sin(2\pi j t))$$

Derivation

$$p_m(t) = \sum_{j=-m}^m c_j e^{2\pi i j t} = c_0 + \sum_{j=1}^m c_j e^{2\pi i j t} + c_{-j} e^{-2\pi i j t}$$

Now use Euler: $e^{i\theta} = \cos\theta + i\sin\theta$; $e^{-i\theta} = \cos\theta - i\sin\theta$ with $\theta = 2\pi j t$

$\Rightarrow$ Then one pos/neg pair becomes:

$$\begin{aligned} & c_j (\cos\theta + i\sin\theta) + c_{-j} (\cos\theta - i\sin\theta) \\ &= \underbrace{(c_j + c_{-j})}_{a_j} \cos\theta + i \underbrace{(c_j - c_{-j})}_{b_j} \sin\theta \end{aligned}$$

$\Rightarrow$ Therefore $p_m(t) = \frac{a_0}{2} + \sum_{j=1}^m (a_j \cos(2\pi j t) + b_j \sin(2\pi j t))$ follows.

! We need $a_j, b_j \in \mathbb{R}$:

Let $c_j = \alpha_j + i\beta_j$ and $c_{-j} = \gamma_j + i\delta_j$ from $a_j = c_j + c_{-j}$
we get:

$\Rightarrow a_j = (\alpha_j + \gamma_j) + i(\beta_j + \delta_j)$; for a_j to be real $\beta_j + \delta_j = 0$.
must hold

$$\Rightarrow \delta_j = -\beta_j \quad \text{I}$$

Now use $b_j = i(c_j - c_{-j})$ we get:

$$\Rightarrow b_j = i[(\alpha_j - \gamma_j) + i(\beta_j - \delta_j)]$$

$\Rightarrow b_j = -(\beta_j - \delta_j) + i(\alpha_j - \gamma_j)$; for b_j to be real $\alpha_j - \gamma_j = 0$.

$$\Rightarrow \gamma_j = \alpha_j \quad \text{II}$$

Put I and II together:

$$\Rightarrow c_{-j} = \alpha_j - i\beta_j = \overline{c_j} \Rightarrow \text{for all } c_{-j} = \overline{c_j}$$

So $a_j, b_j \in \mathbb{R} \Rightarrow c_{-j} = \overline{c_j}$

complex conjugate
of c_j

observe they have
the same magnitude!

Definition 3.1.3 - $L^2(0,1)$

We define $L^2(0,1) = \{f : (0,1) \rightarrow \mathbb{C} : \|f\|_{L^2(0,1)} < \infty\}$

In words: $L^2(0,1)$ contains functions whose squared magnitude is integrable

$$\int_0^1 |f(t)|^2 dt < \infty$$

Why is this even introduced? Because in Fourier analysis we want to treat whole functions like vectors

For an ordinary vector we can talk about; length, direction, orthogonality and we want analogous concepts for functions.

The intuition is that functions in $L^2(0,1)$ behave like vectors in vector space

Scalar product in $L^2(0,1)$

For vectors dot product is a sum of componentwise products.

For functions there are continuously many components, one for each t , so the sum becomes an integral.

$$\langle g, f \rangle_{L^2(0,1)} = \int_0^1 \overline{g(t)} f(t) dt$$

if function is complex this is complex conjugate if not just $g(t)$.

and

$$\|f\|_{L^2(0,1)} = \sqrt{\langle f, f \rangle} = \sqrt{\int_0^1 |f(t)|^2 dt}$$

Fourier Basis functions $\phi_k(t) = e^{2\pi i k t}$

$$\begin{aligned} \langle \phi_k(t), \phi_j(t) \rangle &= \int_0^1 \phi_k(t) \cdot \overline{\phi_j(t)} dt \\ &= \int_0^1 e^{2\pi i k t} \cdot e^{-2\pi i j t} dt = \int_0^1 e^{2\pi i (k-j)t} dt \end{aligned}$$

Case 1: $k=j$; $e^{2\pi i (k-j)t} = e^{2\pi i 0 t} = 1 \Rightarrow \langle \phi_k(t), \phi_j(t) \rangle = \int_0^1 1 dt = 1$

(case 2: $k \neq j$)

$$\text{We integrate } \langle \phi_k(t), \phi_j(t) \rangle = \int_0^1 e^{2\pi i(k-j)t} dt = \left[\frac{e^{2\pi i(k-j)t}}{2\pi i(k-j)} \right]_0^1$$

$\Rightarrow$ This is equal to $\frac{e^{2\pi i(k-j)} - 1}{2\pi i(k-j)}$ and since $k-j \in \mathbb{Z}$

we have $e^{2\pi i(k-j)} = 1$, hence $\langle \phi_k(t), \phi_j(t) \rangle = 0$ for $k \neq j$

$$\Rightarrow \langle \phi_k, \phi_j \rangle = \begin{cases} 1 & k=j \\ 0 & k \neq j \end{cases} \quad \dots \text{orthogonality}$$

Reminder: Euler identities

$$\cos(\theta) = \frac{1}{2}(e^{i\theta} + e^{-i\theta}) \quad \text{and} \quad \sin(\theta) = \frac{1}{2i}(e^{i\theta} - e^{-i\theta})$$

Using this we can also show that for $k \neq j$ $\langle \sin(2\pi jt), \sin(2\pi kt) \rangle$, $\langle \cos(2\pi jt), \cos(2\pi kt) \rangle$ are all 0, hence orthogonal.

How do we find our coefficients?

In vector sense $v = ae_1 + be_2$ with orthogonal unit vectors e_1, e_2

To find a , we take the scalar product with e_1 :

$$\begin{aligned} \langle v, e_1 \rangle &= a \underbrace{\langle e_1, e_1 \rangle}_{=1} + b \underbrace{\langle e_2, e_1 \rangle}_{=0} \\ \langle v, e_1 \rangle &= a \end{aligned}$$

For Fourier coeffs:

$$f(t) \approx \frac{a_0}{2} + \sum_{k=1}^m (a_k \cos(2\pi kt) + b_k \sin(2\pi kt))$$

and say we want to find a_j .

$\Rightarrow$ We multiply the whole function by $\cos(2\pi jt)$. From orthogonality, every cosine with frequency $k \neq j$ and every sine disappears. Only the cosine with frequency j survives

$\Rightarrow$ We integrate over $[0, 1]$

$$\int_0^1 f(t) \cos(2\pi j t) dt$$

$$= \int_0^1 \left[\frac{a_0}{2} + \sum_{k=1}^n (a_k \cos(2\pi k t) + b_k \sin(2\pi k t)) \right] \cos(2\pi j t) dt$$

$$= \frac{a_0}{2} \int_0^1 \cos(2\pi j t) dt + \sum_{k=1}^n a_k \int_0^1 \cos(2\pi k t) \cos(2\pi j t) dt + \sum_{k=1}^n b_k \int_0^1 \sin(2\pi k t) \cos(2\pi j t) dt$$

Orthogonality does the work for us, we are left with:

$$\int_0^1 f(t) \cos(2\pi j t) dt = a_j \int_0^1 \cos^2(2\pi j t) dt = 1/2$$

$$\Rightarrow a_j = 2 \int_0^1 f(t) \cos(2\pi j t) dt$$

Similarly,

$$b_j = 2 \int_0^1 f(t) \sin(2\pi j t) dt$$

and

$$a_0 = 2 \int_0^1 f(t) dt$$

Fourier coefficient = strength of one frequency inside the function

When you use Euler identities with our real-valued formula, you will get:

$$a_k \cos(2\pi k t) + b_k \sin(2\pi k t) = \frac{a_k - ib_k}{2} e^{2\pi i k t} + \frac{a_k + ib_k}{2} e^{-2\pi i k t}$$

$$= \hat{f}(k) e^{2\pi i k t} + \hat{f}(-k) e^{-2\pi i k t}$$

$$\text{and define } \hat{f}(0) = \frac{a_0}{2}$$

$$\text{Hence } f(t) = \hat{f}(0) + \sum_{k=1}^{\infty} (\hat{f}(k) e^{2\pi i k t} + \hat{f}(-k) e^{-2\pi i k t})$$

$$= \sum_{-\infty}^{\infty} \hat{f}(k) e^{2\pi i k t}$$

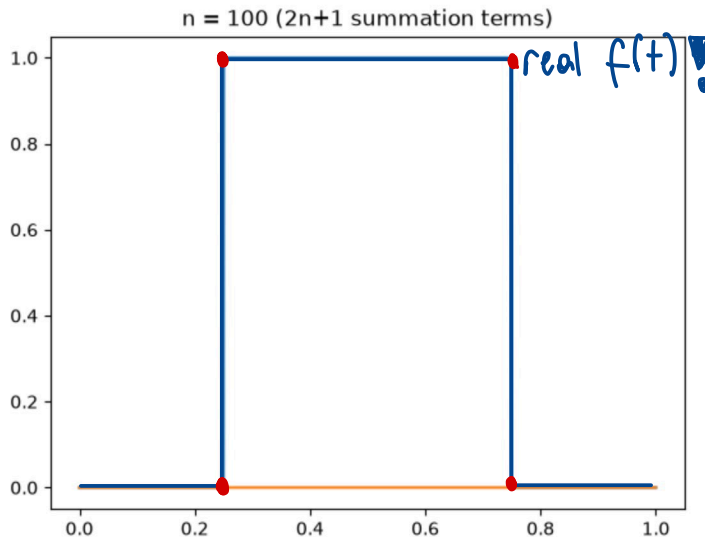
$$\text{And } \hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt$$

we used definition of a_k and b_k and to get this

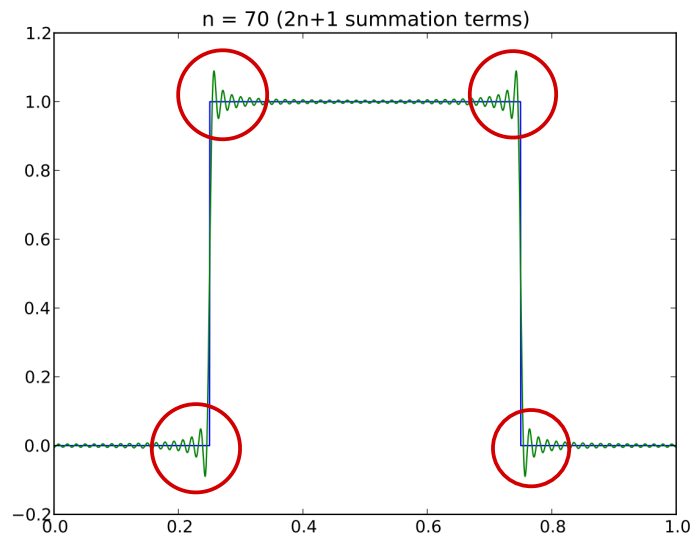
Code expert: Gibbs - Phänomen

We are approximating the function $f(t) \begin{cases} 1 & , a \leq t \leq b \\ 0 & \text{otherwise} \end{cases}$

for this exercise $a = 0.25$ and $b = 0.75$.



We have discontinuities; near this jump discontinuities, a Fourier series develops oscillations and overshoots $\rightarrow$ Gibbs



Coding Tips

- $$b - a + \frac{1}{\pi} \sum_{k \neq 0} e^{-ikc} \underbrace{\frac{\sin kd}{k}}_{e^{ik(2\pi t - c)}} e^{i2\pi kt}, \quad t \in [0, 1],$$

$$c = \pi(a+b); \quad d = \pi(b-a)$$

already defined

- t is x in the code

Discussion

$$\hat{f}(k) = \int_0^1 f(t) e^{-2\pi i k t} dt \quad \text{and} \quad \hat{f}(k) = \text{strength of frequency } k \text{ inside } f$$

At the beginning we said larger $|k|$ means very high-freq oscillations.

For a smooth function, we usually do not need much high-freq. content

smooth $f \Rightarrow |\hat{f}(k)|$ decays quickly as $|k| \rightarrow \infty$

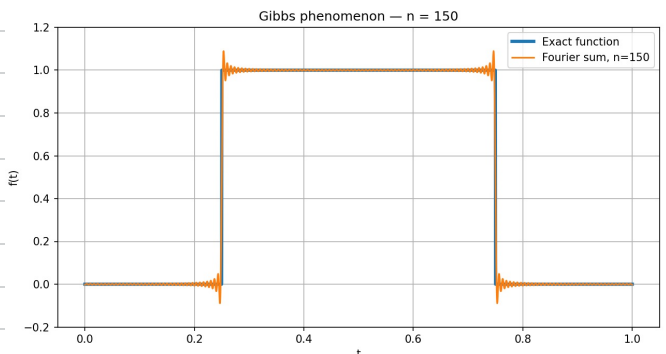
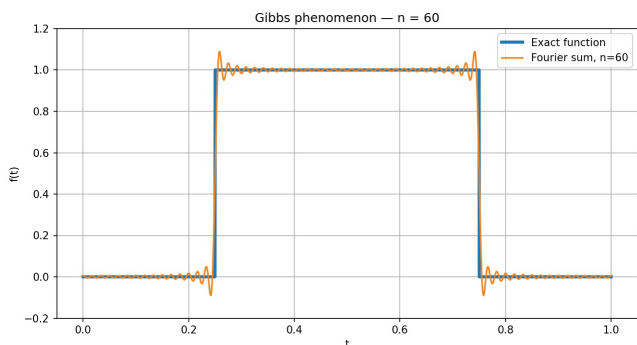
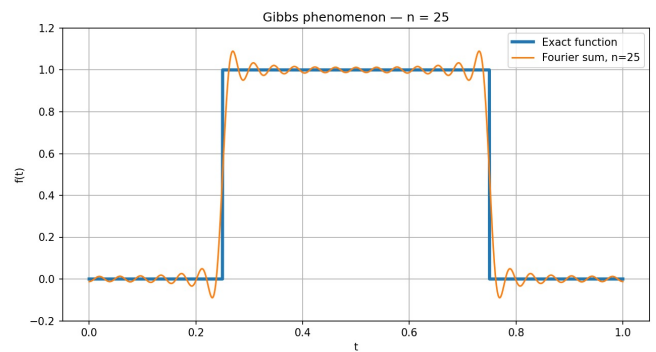
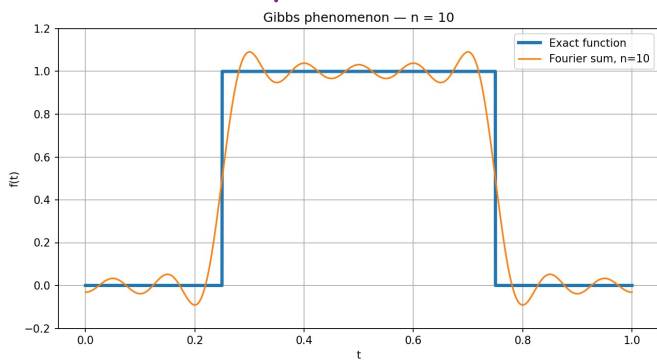
$\rightarrow$ faster convergence

The script makes this concrete:

$$\int_0^1 |f^{(n)}(t)| dt < \infty \Rightarrow \hat{f}(k) = O(k^{-n})$$

Now our $f(x)$ in this exercise is not smooth at all, therefore we should expect slow decay of Fourier coeffs and therefore slow convergence of the Fourier series.

$\rightarrow$ Code example 1



The height of over/undershooting doesn't change, but the area where we have disruptions gets smaller with increasing n .

Until now we assumed that we know $f(x)$

Now in numerical applications we normally do not know $f(x)$ everywhere, we only have measurements:

$$f(t_0), f(t_1), \dots, f(t_{N-1})$$

Usually these are taken at equally spaced points

$$t_j = \frac{j}{N}, \quad j = 0, \dots, N-1$$

We need to find Fourier coefficients given only N samples.

Remark 3.1.19 Approximating Fourier Coefficients

We approx earlier integral by a sum:

$$\hat{f}(k) = \frac{1}{N} \sum_{j=0}^{N-1} f(t_j) \cdot e^{-2\pi i k t_j} \quad ; \quad t_j = \frac{j}{N}$$

$$= \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) e^{-2\pi i k j / N}$$

Intuition: Before we multiply $f(t)$ with one wave and integrate the coefficient. Now we multiply the sampled values with one sampled wave and sum

Definition 3.2.1. Roots of Unity

To simplify the notation we define $\omega_N = e^{-2\pi i / N}$, then $e^{-2\pi i k} = \omega_N^k$ hence:

$$\hat{f}_N(k) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(\frac{j}{N}\right) \omega_N^{kj}$$

What is ω_N geometrically?

→ Code example 2

It is the complex number on the unit circle with angle $-2\pi/N$. So its powers:

$$1, \omega_N, \omega_N^2, \dots, \omega_N^{N-1}$$

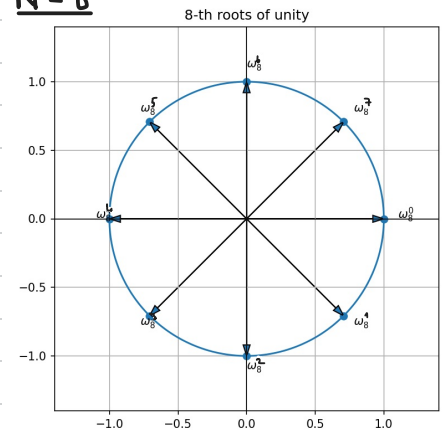
corresponds to just rotation by $2\pi/N$ clockwise.

Because these points are perfectly symmetric around the origin, their vector sum is zero:

$$1 + \omega_N + \omega_N^2 + \dots + \omega_N^{N-1} = 0$$

Again we have different frequencies cancelling each other out.

N=8



Definition 3.2.5 - Discrete Fourier Transform ★

Let $y_j = f(t_j)$ be our sampled values.

Then DFT is

$$c_k = \sum_{j=0}^{N-1} y_j \omega_N^{kj} \quad \text{for } k=0, \dots, N-1$$

What does it do?

Input $y = (y_0, y_1, \dots, y_{N-1})$... signal in time domain

Output $c = (c_0, c_1, \dots, c_{N-1})$... signal in frequency domain

Intuition

Think of time domain as: "what value does my signal f has at each moment"

$\Rightarrow y_0 = \text{signal now}$; $y_1 = \text{signal a little later}$...

↳ DFT then answers what frequencies are in this signal.

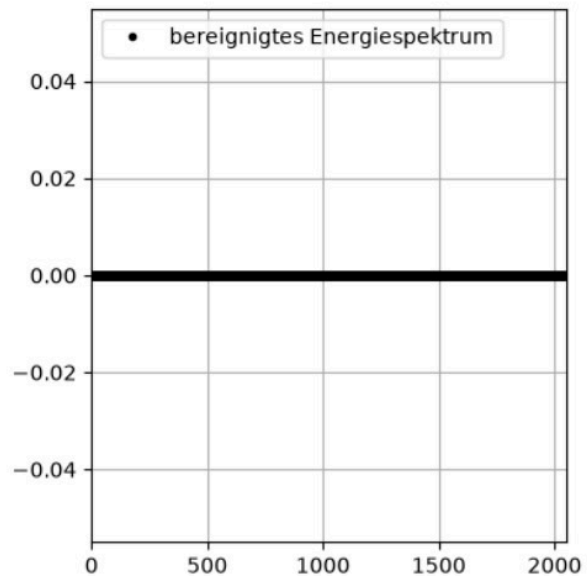
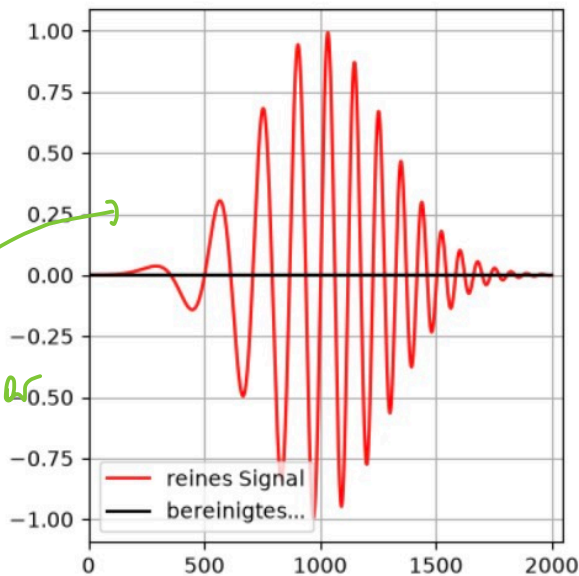
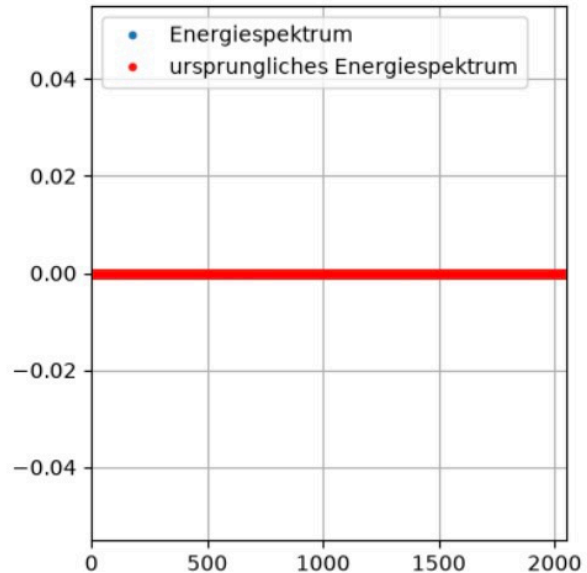
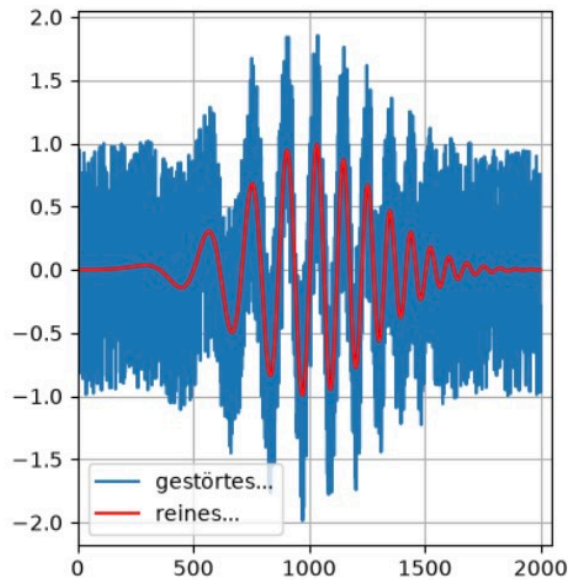
Code expert: Filtering

We have a clean signal f , but the measured data contains noise:

$$y = f + \text{noise}$$

The idea is:

go to freq. domain $\rightarrow$ remove weak frequencies $\rightarrow$ transform back



here you should see function similar to red.

You should see the right side changed after implementation.

Cooling Tips

- you should keep frequencies that have a higher power P_k than $\Gamma + P_{\max}$
- research `np.fft.fftfreq()` and `np.fft.ifftfreq()`
- power/energy spectrum looks at $|\hat{f}(k)|^2$

Discussion

From Fourier Approx. to Trigonometric Interpolation

Take equally spaced points again $t_j = \frac{j}{N}$ $j=0, 1, \dots, N-1$.

We compute the discrete Fourier coefficients, then construct the corresponding trigo. polynomial.

It turns out that $p(t_j) = f(t_j)$, so the polynomial passes exactly through all measurements.

So Fourier Approx on uniform samples $\Rightarrow$ trigonometric interpolation

Zero - Padding

Suppose we have n samples, we construct a trigonometric interpolant out of these.

Now we want to evaluate this at $N > n$ points. We do not want to calculate Fourier sum at every point.

$$S_N(t) = \sum_{k=-N}^N \hat{f}(k) e^{2\pi i k t}$$

So instead we use zero-padding.

Idea Our interpolant only contains the frequencies represented by the original n coefficients

$\Rightarrow$ If we use a Fourier vector of length N , the additional high-freq coeffs must be 0.

So [old coefficients] become [pos-freqs, 0, ..., 0, neg-freqs]

Then we compute inverse transform to evaluate the same $p(t)$ at a finer grid. No new frequency info added.

$\Rightarrow$ Code example 3 $M=8, 16, 128$

Inverse transform just does $y_j = \sum_{k=0}^{n-1} c_k e^{2\pi i j k / N}$

with zero-padding $y_j = \sum_{k=1}^{N-1} c_k e^{2\pi i j k / N}$

Code expert: Fourier Interpolation

Coding Tips

- Both evalDtrip and convtrip is in the script
- For TODO3 look at Example 3.5.4 and to the input of convtrip, something is missing.

Code expert: Approximation von f und f'

Frequency ordering in the FFT:

Suppose $N = 8$

Mathematically we would like to think about the Fourier frequencies in the natural order

$-4, -3, -2, -1, 0, 1, 2, 3 \rightarrow \text{np.arange}(-4, 4)$

But Numpy doesn't store FFT coefficients in this order, instead it uses the order

array index	0	1	2	3	4	5	6	7
stored coeff	c_0	c_1	c_2	c_3	c_{-4}	c_{-3}	c_{-2}	c_{-1}
actual frequency	0	1	2	3	-4	-3	-2	-1

$\Rightarrow$ fftshift gets us this human-friendly version!

Normally we don't care about this at all, as when using FFT/IFFT in Numpy it knows how it orders things and work well

BUT now we need differentiation in evalDtrip

Fourier Differentiation

$$p(t) = \sum_k c_k e^{2\pi i k t}, \text{ differentiate it term by term } p'(t) = \sum_k c_k \frac{d}{dt} e^{2\pi i k t}$$

differentiation
amplifies high-freq.
components

$$= \sum_k (2\pi k) c_k e^{2\pi i k t}$$

Differentiate in time/space $\Leftrightarrow$ multiply coefficient c_k by $2\pi i k$ ★

$(2\pi i k) \cdot$
 $\swarrow$
 $\text{corr. } k\text{'s}$

array index	0	1	2	3	4	5	6	7
stored coeff	c_0	c_1	c_2	c_3	c_{-4}	c_{-3}	c_{-2}	c_{-1}
actual frequency	0	1	2	3	-4	-3	-2	-1

So don't multiply blindly, first shift to make it easy!

Coding Tips

- Compute derivatives as I explained
- Perform zero-padding again in the same manner as before
- $n // 2$ means integer division by 2, so it is rounded down.

Nyquist Limit

Again suppose we take $t_j = \frac{j}{N}$ over an interval of length 1.

The sampling rate is N . ($f_s = \frac{1}{\Delta t}$, Δt ... time between consec. samples)

Then the largest distinguishable frequency is $N/2$.

Extra: Why?

For frequency k , $e^{2\pi i k t_j} = e^{2\pi i k j / N}$, now compare k and $k+N$
 $\Rightarrow e^{2\pi i (k+N)j / N} = e^{2\pi i k j / N} e^{2\pi i j}$ and as $j \in \mathbb{Z}$ we know $e^{2\pi i j} = 1$
 $\Rightarrow e^{2\pi i (k+N)j / N} = e^{2\pi i k j / N}$

So the samples literally cannot distinguish frequencies separated by N (= sampling rate)

This is ALIASING, example $N=8$:

$k=1 \Rightarrow e^{2\pi i j / 8}$ and $k=9 \Rightarrow e^{2\pi i j 9 / 8} = e^{2\pi i j / 8} e^{2\pi i j} = e^{2\pi i j / 8}$

The computer cannot know which one generated the dots, as for different j 's they are same.

We know for real-valued signals, positive and negative freqs are paired.

So as we move beyond $\frac{N}{2}$, discrete samples start looking like a lower negative freq.

$$\rightarrow -\frac{N}{2}, \dots, \frac{N}{2} - 1$$

Question: We said $y \xrightarrow{\text{fft}} c$ but in code expert $c = \text{np.fft.ifft}(y)$
What is the reason for this?

not important, just so there is
no confusion about this